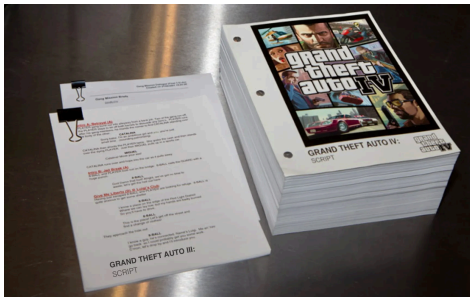


Guía para escritura de GDDs

GDD Colombia. Escrito por Andrés López y Daniel Durán

Un GDD (*Game Design Document* en inglés) es un documento elaborado por un desarrollador o equipo para describir la visión e ideas de un videojuego. Su objetivo principal es comunicar la visión del proyecto a todo el equipo, aterrizar ideas y presentar el proyecto a terceros de manera sencilla y fiable. No se debe confundir el GDD como documento con el nombre del grupo estudiantil, que también es GDD.

Hay dos métodos para elaborar un GDD:



Monolítico:

- Un documento tradicional, redactado en un procesador de texto
- Son enciclopédicos, extensos y están diseñados para su publicación en internet y una lectura detallada
- Suelen escribirse durante o después del desarrollo, y no es raro que sean redactados por terceros en lugar de los propios desarrolladores

Vivo o dinámico:

- Un documento no lineal creado en una herramienta colaborativa (HackMD, Notion, Collanote, etc.)
- Se actualiza a la par del desarrollo y sirve de apoyo y consulta
- Se acompañan de enlaces, documentos breves y contenido multimedia
- Se elaboran al inicio o durante el desarrollo por parte de los mismos desarrolladores

Nosotros usamos ambas formas, y ambas son útiles. Sin embargo, para el **checkpoint 1** solicitamos un **GDD monolítico**. El GDD vivo se redacta durante el desarrollo y no sigue una estructura fija, aunque esta guía resulta igualmente útil para ese tipo de documentos.

¿Por qué pedimos un GDD monolítico?

Un GDD monolítico se redacta en un solo documento exhaustivo. suele ser extenso, poco atractivo de leer, tiende a desactualizarse y no siempre se adapta al proceso creativo de un proyecto. Entonces, ¿por qué lo utilizamos?

La razón principal para seguir esta estructura es que resulta perfecta para **argumentar tu proyecto**. Te obliga a responder preguntas clave mientras el GDD registra las respuestas. Tu documento no tendrá 100 páginas, tal vez unas 20 a 50 como máximo. Esto es perfectamente manejable en la práctica, por lo que no es un archivo infinito e inútil, sino un documento accesible y orientado a la planificación.

En todo caso, se recomienda elaborar un GDD monolítico en un tiempo y extensión razonables antes de iniciar el proyecto. Una vez comiences el desarrollo, puedes migrar tu GDD a un formato dinámico que te permita iterar y adaptarlo a tus avances.

Estructura de un GDD

1. Resumen

- Sinopsis
- Género
- Público objetivo

2. Desarrollo

- Herramientas a usar
- Plataformas objetivo
- Flujo de trabajo Opcional para nuevos proyectos

3. Diseño

- Pilares de diseño
- Identidad y puntos clave
- Experiencia del jugador
- Game loop
- Progresión
- Sistemas Opcional para nuevos proyectos

4. Contenido

- Mecánicas
- Aprendizaje del jugador
- Estilo artístico
- Música y sonido
- Mundo y escenarios
- Historia y personajes
- Interfaz

5. Referencias

- Inspiraciones
- Investigación de mercado Opcional para nuevos proyectos
- Contenido preliminar Opcional para nuevos proyectos
- Anexos (opcional)

Cosas que no van en este GDD (y normalmente en ninguno)

- Detalles de implementación: van en la documentación
- Contenido planeado para MVP: va en la documentación
- Cronogramas: siempre se incumplen y desactualizan
- Alcance planeado: no es relevante y quizá sea irrealista
- Requerimientos técnicos para el juego: difícil de dimensionar y poco relevante

¿En que orden se diligencia?

Es recomendable diligenciar primero la sección «Referencias» para obtener información externa, luego las demás secciones en orden. Las secciones están diseñadas para que puedas diligenciarlas con la información de secciones anteriores, por lo que ir en orden puede beneficiarte.

En todo caso, puedes completar el documento en el orden que te parezca más relevante para tí.

Contenido de un GDD

La estructura que recomendamos consta de 5 secciones: resumen, desarrollo, diseño, contenido y referencias. Cada una busca abordar un aspecto diferente y ofrecer una visión integral de tu proyecto. Es fundamental destacar que un GDD **no hace las veces de documentación**; por lo tanto, los detalles minuciosos e implementaciones específicas no van en este documento. Un GDD se enfoca en las características esenciales del proyecto.

1. Resumen

Sinopsis

Un resumen de 1 o 2 frases que explique de qué va tu juego y por qué es divertido o interesante. Escríbelo como si lo fueras a incluir en la descripción de Steam o a explicárselo a un amigo.

Ejemplo Expedition 33 en Steam

«Guía a la expedición 33 en su misión de acabar con la Pintora para que no vuelva a pintar la muerte. Explora un mundo inspirado en la Francia de la Belle Époque y lucha contra rivales únicos en este juego de rol por turnos con mecánicas en tiempo real.»

Género

¿Cuál es el género de tu videojuego? Esto define directamente una gran cantidad de características y lo enmarca en una categoría específica. En internet puedes encontrar listas de géneros que te servirán de guía. Un juego puede pertenecer a más de un género a la vez, o ser parte de géneros más específicos (ej. «Horror de mascotas» o «Mundo abierto de vóxeles»).

Ejemplo Grand Theft Auto V

GTA V (y toda la saga GTA) es un juego de acción, aventura y mundo abierto. De hecho, es común que la acción y la aventura vayan de la mano.

Público objetivo

Tu juego **jamás** será para todo el mundo. Definir qué tipo de persona quieres que juegue tu juego te ayudará a comprender qué diseño adoptar y qué contenido incluir. Describe motivaciones, gustos, edades, géneros, e incluso inspiraciones y características que atraigan a las personas indicadas.

Tu público objetivo no es un límite estricto: técnicamente cualquiera podría jugar tu título, pero no a todos les va a gustar (estos últimos quedan fuera de dicho público). Asimismo, el público objetivo aclara quiénes no deberían jugar: un juego +18 no debe ser jugado por menores de edad, mientras que uno enfocado en preescolares resulta inadecuado o irrelevante para adultos.

Ejemplo Minecraft

Minecraft es uno de los videojuegos más universales de la actualidad. Sin embargo, eso no le impide delimitar su público objetivo a:

- Mayores de 8 años (especialmente entre 12 y 24 años)
- Personas que destacan por ser creativas y resilientes
- Educadores y estudiantes

2. Desarrollo

Herramientas a usar

Lista, y describe si es necesario, las herramientas, aplicaciones o software que se utilizarán para producir el juego. Esto abarca:

- Motor de videojuegos y lenguajes de programación
- IDE/Editor de texto y código
- Gestión de código y recursos multimedia
- Modelado y herramientas 3D
- Ilustración y edición de fotos
- Herramientas de productividad
- Editores de audio y música
- Librerías o software propio, si aplica

Existen algunas obviedades que puedes omitir en esta sección:

- Git para control de versiones. Lo menos obvio sería usar otra cosa, como Mercurial o Apache Subversion
- WhatsApp y Discord para comunicación, pues son los canales habituales
- Hardware, tanto el esencial para el desarrollo como para el testeo

Si te parece relevante, puedes explicar brevemente porque has elegido las herramientas que usarás. Te ayudará a comprender tu decisión a futuro y evitar migraciones/cambios de software innecesarios.

Ejemplo Slay the Spire 2

Slay the Spire 2 fue desarrollado en Unity + C# y luego migrado a Godot + C#. También se utilizaron librerías propietarias de Mega Crit. No hay información más detallada sobre el resto de herramientas usadas; en tu caso, puedes elegir las que consideres adecuadas.

Plataformas objetivo

¿En qué plataformas y dispositivos estará disponible el juego? Cada plataforma representa un mercado, un público objetivo, necesidades particulares y un estilo propio. Esto influye de manera determinante en lo que tu juego puede ofrecer. Por ejemplo, un juego competitivo o de pago probablemente no sea el más adecuado para teléfonos móviles, mientras que uno hipercasual no se adapta bien a consolas. Cada plataforma implica un paradigma de juego distinto.

Ejemplo HALO: Combat Evolved

HALO: Combat Evolved fue lanzado oficialmente para:

- Xbox
- Windows
- macOS X

Flujo de trabajo Opcional para nuevos proyectos

¿De qué manera van a trabajar como equipo o individuo? ¿Cuáles son sus prioridades (pruebas rápidas, retroalimentación, distribución de roles)? Definir esto permite despejar dudas y evitar bloqueos en el grupo. Algunas nociones que se pueden discutir son:

- Repartición de tareas y manejo del proyecto: ¿Quién hace qué? ¿Todos tienen acceso a todo? ¿Como fluyen los recursos desde su autor hasta el producto final?

- Integración del trabajo: ¿Como implementan todo? ¿Como evitan/solucionan conflictos?
- Forma de iteración: ¿Iteran rápido o se esfuerzan en un solo intento hasta que quede bien? ¿Qué hacen con los prototipos?
- Manejo del feedback: ¿Piden retroalimentación a terceros? ¿Como se gestiona la opinión del equipo?

No es obligatorio responder cada pregunta. Solo las que sean relevantes, y las que quieras añadir que valga la pena tener en cuenta.

Ejemplo Deltarune

Deltarune se desarrolla principalmente a partir del prototipado y las ideas de Toby Fox, refinadas por Temmie Chang e implementadas con la asistencia de Toby y otros desarrolladores (algunos de ellos pertenecientes a 8-4 para las versiones de consolas y en japonés).

Por lo que se conoce, su proceso implica un trabajo directo y detallado enfocado en el contenido final, sin iteraciones masivas ni retroalimentación directa de la comunidad.

3. Diseño

Pilares de diseño

¿Qué rige el diseño de tu juego? Como si fuesen mandamientos para el equipo y el contenido del juego, los pilares de diseño son principios simples y claros que orientan el desarrollo. A diferencia del contenido, los pilares de diseño no suelen cambiar a lo largo del proyecto, y plasman las ambiciones, la filosofía y las necesidades de tu equipo.

Los pilares de diseño son útiles para evaluar y guiar decisiones, evitando perder el rumbo o arruinar el proyecto con ideas incompatibles. Definirlos desde el principio ayuda a concretar los objetivos del grupo y la forma en que trabajarán.

Estos pilares pueden ser de dos tipos: prácticos (para orientar la jugabilidad y el contenido) y filosóficos (para definir la mentalidad del equipo respecto al proyecto). Se recomiendan entre 3 y 5 principios bien explicados, combinando ambos enfoques si lo deseas.

Ejemplo Hades

Hades, de Supergiant Games, se fundamenta en la fusión entre los roguelikes exigentes y una narrativa progresiva y dinámica. Sus principios son principalmente prácticos:

- Usar el fracaso como motor de la narrativa
- Minimizar el contenido repetitivo
- Combate rápido, frenético y súper dinámico
- Estética llamativa y dinámica
- Música adaptativa e inmersiva

Identidad y puntos clave

¿Qué es lo que hace único tu juego? Plantear estos puntos de manera plenamente innovadora o revolucionaria no es obligatorio, pero resulta esencial definir algo para destacar y aportarle personalidad al proyecto.

Antes de definirlos, evita el plagio e investiga el mercado actual y tus intereses en busca de novedades. También es importante que reflejen parte de tu estilo o sello personal, lo cual facilita interiorizar la idea y potenciar la creatividad.

Ejemplo Índigo Park: Chapter 1

Índigo Park es un juego de horror de mascotas (al estilo FNAF) creado por Mason Myers (UniqueGeese). Uno de sus puntos más identitarios es contar con una mascota aliada que guía tu experiencia, Rambley; además, posee una historia relevante expresada con una ternura que contrasta con el ambiente. Esto subvierte las convenciones del género, donde los personajes suelen ser exclusivamente enemigos sin contexto ni mayor atractivo que los *screamers* o un lore cuestionable.

Experiencia del jugador

¿Qué experiencia se busca generar en el jugador a lo largo del juego? ¿Qué fantasía queremos transmitirle? Usualmente, la experiencia deseada se puede definir como:

- Transmitir una emoción o sensación específica
- Hacer que el jugador interprete a un personaje
- Diseñar una actividad o dinámica particular

Ejemplo Undertale

Undertale es reconocido por subvertir los RPGs tradicionales al introducir el concepto del perdón y la relevancia de las decisiones. La experiencia busca otorgar valor a tus acciones y apelar a la empatía, en lugar de invitar a jugar en «piloto automático».

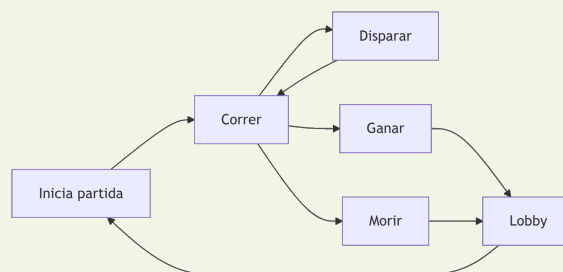
Game loop

Un *game loop* es un diagrama en bucle o una explicación que describe el núcleo de las mecánicas y el orden en que el jugador realiza sus acciones. Esto se considera diseño (y no contenido) porque el *game loop* no depende del contenido, sino que lo rige, y describe más las mecánicas.

El *game loop* está estrechamente relacionado con el género de tu videojuego; por ello, puedes empezar revisando la estructura típica de dicho género y adaptarla a tus necesidades. Puedes elaborar los diagramas en Mermaid, Miro, Canva o la herramienta que prefieras, o bien explicarlo mediante texto si resulta más claro.

Ejemplo

Los juegos de disparos/FPS (como Fortnite) suelen compartir el mismo bucle básico:



Otras dinámicas, como las zonas seguras o el botín (*loot*), dependen del juego en particular.

Progresión

¿Cómo avanza el jugador en tu juego? ¿Cuáles son sus objetivos a corto y largo plazo? ¿Están alineados con la fantasía que deseas transmitir? Al igual que el *game loop*, la progresión suele definirse con base en el género seleccionado. Por supuesto, la progresión también se vincula al contenido y a la narrativa del juego, aunque no los determina *per se*.

Además de los objetivos, también se discuten el ritmo, la dificultad y las dinámicas de avance del juego: ¿Tu juego es frénético, calmado o abrupto? ¿Cuán difícil pretendes que sea al avance? ¿Cómo se mide el progreso del jugador?.

Ejemplo Saga Pokémon

En Pokémon, los objetivos a corto plazo consisten en obtener medallas y fortalecer a tu equipo, con el fin supremo de enfrentar al Alto Mando y al campeón de la región.

Sistemas Opcional para nuevos proyectos

¿Qué sistemas incluye tu juego y cómo impactan en la experiencia del jugador? Los sistemas suelen abarcar:

- Economía y gestión de recursos
- Clasificaciones, escalas y puntuaciones
- Sistemas de batalla, combate, y para la acción en general

No es necesario incluir implementaciones técnicas ni detalles minuciosos (como las fórmulas matemáticas/físicas o los precios de artículos específicos), ya que no son relevantes en este punto. Esos detalles corresponden a la documentación técnica; aquí solo debes explicar cómo funcionan y por qué son importantes.

Algunos ejemplos evidentes que se pueden omitir:

- Interfaz de usuario (es obvio y va más adelante)
- Sistemas de la implementación/motor (ej.: uso de *GameObjects* de Unity)
- Sistemas indispensables (movimiento, guardado-carga)

Puedes incluirlos si son especialmente innovadores o identitarios para tu juego; de lo contrario, es recomendable omitirlos.

Ejemplo Geometry Dash

Geometry Dash contiene (y destaca) por estos sistemas:

- Creación y jugar de niveles con música de Newgrounds o propia
- Dos modos de juego: normal y plataforma (desde la 2.2)
- Personalización de los vehículos
- Monedas, gemas, estrellas, llaves... todas con sus propios usos
- *Gaunlets* y *map packs*

4. Contenido

Mecánicas

¿Qué acciones puede realizar el jugador? Describe brevemente las mecánicas principales en términos de verbos o acciones. Las mecánicas son el núcleo de tu juego; aunque de forma individual no sean totalmente innovadoras, la combinación entre ellas o su enfoque sí puede serlo.

Los buenos juegos suelen mezclar su narrativa directamente con las mecánicas, y ambas se complementan. Por ende, piensa sobre cómo tus mecánicas conectan con la historia y con el jugador. El estilo narrativo y la justificación/lore serán tus aliados en esto.

Ejemplo Undertale

En Undertale (y Deltarune) destacan:

- El sistema de piedad (LOVE) que altera la historia y cuestiona al jugador
- El *bullet hell* (esquivar balas con patrones) heredado de Touhou, en este caso con la alma (corazón)
- Una historia con sus rutas basada en las acciones del jugador

Estas mecánicas dirigen y conectan la narrativa con el jugador y sus decisiones. Por eso Undertale ha sido tan exitoso en su objetivo.

Aprendizaje del jugador

¿Cómo aprende el jugador a jugar tu juego? Idealmente no deberías necesitar tutoriales ni pistas, pero si es el caso, indícalo. Puedes comenzar por indicar las mecánicas asociadas al aprendizaje del jugador, o apelar a principios de diseño (si no los has indicado antes). El objetivo es hacer un juego que no trate a los jugadores como tontos (la infame pintura amarilla, por ejemplo).

Ejemplo Mario Bros.

Mario Bros. no tiene tutorial. En cambio, te pone en el primer nivel, con la capacidad de progresar adecuadamente por los obstáculos posibles (Gumbas, plataformas, Koopas, huecos, plantas) y las mecánicas (bloques especiales, pisar gumbas, entrar en tubos), etc.

Estilo artístico

El estilo artístico define el aspecto visual del juego. Puede ser arte vectorial, *low poly*, *pixel art*, ilustrado, realista, entre otros. Se recomienda especificar el estilo en detalle (por ejemplo, no basta con decir «pixel art», sino detallar si es 8-bit, Hi-res o rotoscopia 3D, u otro diferente).

Ejemplo Deltarune

Deltarune utiliza *pixel art*, pero es necesario ser más preciso. Sigue una técnica tradicional, con resolución baja-media y un estilo dinámico y moderno que conserva sus raíces retro.

Al compararlo con Undertale (del mismo autor), notarás diferencias significativas en la dirección de arte. Por ello es importante especificar exactamente qué tipo de arte se busca.

Música y sonido

¿Qué estilo musical se utilizará? ¿Cómo influye la música en otros aspectos del juego y cómo contribuye a la experiencia del jugador? Al igual que el resto del contenido, debe prestarse para la narrativa y la

experiencia del jugador. Por lo que indicar la forma en la que la música es diseñada, y la forma en la que conecta con los objetivos del juego es esencial.

Ejemplo Devil May Cry

La música de Devil May Cry es enérgica y emocionante, lo que genera en el jugador un sentimiento de adrenalina que lo motiva a combatir enemigos y realizar combos.

Mundo y escenarios

¿En qué mundo se desarrolla el juego y cuáles son sus características principales? ¿Qué acontecimientos marcan el contexto del entorno? Además del diseño de escenarios, se puede incluir la construcción de mundo (*worldbuilding*) o el trasfondo (*lore*), aunque este último también puede formar parte de la sección de historia.

Procura no añadir detalles excesivamente específicos o irrelevantes.

Ejemplo Devil May Cry 3

Devil May Cry 3 se ambienta en un universo donde coexisten demonios y humanos. Tiempo atrás, el demonio Sparda selló el portal entre ambos mundos. En el juego, Vergil (uno de los hijos que Sparda tuvo con una humana) busca abrir nuevamente el portal para obtener más poder, mientras que su hermano gemelo, Dante, intenta detenerlo.

Historia y personajes

¿Cuáles son los puntos centrales de la narrativa del juego? ¿Quiénes son los personajes principales? Puedes explicar ambos juntos (historia completa y los personajes como contexto, como en el ejemplo) o separarlos. Procura igualmente que no caigas en descripciones innecesarias o extensivas.

Ejemplo God of War

God of War narra la historia de Kratos, un guerrero espartano que vendió su alma a Ares, dios de la guerra, para obtener la victoria en batalla. Tras ser engañado por Ares y asesinar a su propia familia, Kratos se convierte en el «Fantasma de Esparta» y decide ponerse al servicio de los dioses para liberar su mente de esos atormentadores recuerdos.

Durante el juego, Kratos emprende una búsqueda para obtener la caja de Pandora y derrotar a Ares por encargo de los dioses. Tras diversos giros argumentales, Kratos se enfrenta a Ares y, al vencerlo, ocupa su lugar como el nuevo dios de la guerra.

Interfaz

¿Cómo interactúa el jugador con el juego y cómo recibe la información importante? Considera controles, dispositivos de entrada, menús y elementos gráficos generales (HUD). También puedes detallar el flujo del usuario por la interfaz.

Ejemplo Doki Doki Literature Club & Minecraft

En las novelas visuales como Doki Doki Literature Club, la interacción se basa en tomar decisiones, completar algunos minijuegos y hacer clic para avanzar los diálogos.

En Minecraft, por su parte, el jugador cuenta con un menú de inventario e interactúa rompiendo y colocando bloques o creando herramientas. Asimismo, dispone de indicadores visuales de salud y hambre para gestionar su supervivencia.

5. Referencias

Inspiraciones

¿Qué obras sirven de inspiración para tu proyecto? Con frecuencia se trata de otros videojuegos, pero también pueden ser libros, películas o experiencias personales. Explica qué elementos te inspiran y qué aspectos deseas retomar. En el caso de referentes poco conocidos, es recomendable incluir un enlace o una breve descripción.

Ejemplo Beast Card Clash

Beast Card Clash es uno de los proyectos de GDD más recientes y avanzados. Se inspira fuertemente en el minijuego Card-Jitsu Fuego de Club Penguin, combinando elementos sencillos de mundo abierto y *roguelike*. Su estética *cozy* recuerda a títulos como Animal Crossing.

Investigación de mercado Opcional para nuevos proyectos

Investiga sobre juegos similares al tuyo (en tiendas digitales, opiniones y otras fuentes) e indica:

- ¿Qué juegos encuentras? ¿Cuáles conoces?
- ¿Qué ves en común? Esto te ayudará a evitar temas «quemados»
- ¿Qué sientes que le falta? Esto te dará ideas para tu proyecto
- ¿Qué errores ves en los juegos? Así los evitas o los reformulas
- ¿Cuán populares son estos juegos y sus géneros?

Esta información te será muy útil para contrastar tus ideas con la realidad, obtener inspiración y definir el contenido que le puede gustar a los posibles jugadores.

Contenido preliminar Opcional para nuevos proyectos

Sección para incluir el material existente del proyecto: bocetos, ideas iniciales de contenido, capturas o pruebas de un prototipo, links, etc.

Anexos (opcional)

Notas adicionales o elementos que no encajen en las categorías anteriores.

Bibliografía

Recursos

- [Lista de géneros de videojuegos en Wikipedia](#) (en inglés)
- [Estructura profesional de un GDD por UDIT](#)
- [Documento de diseño de videojuegos - Wikipedia](#)
- [Tutorial para GDDs por Alva Mayo](#)
- [Ejemplos de GDD](#) (en inglés)

Juegos mencionados

- [Expedition 33](#) (2025)
- [Saga GTA](#) (desde 1997)
- [Minecraft](#) (2011)
- [Slay the Spire 2](#) (2026)
- [HALO: Combat Evolved](#) ()
- [Undertale](#) (2015)
- [Deltarune](#) (desde 2018)
- [Hades](#) (2020)
- [Índigo Park: Chapter 1](#) (desde 2024)
- [Fortnite](#) (2017)
- [Saga Pokémon](#) (desde 1996)
- [Geometry Dash](#) 2013
- [Hollow Knight](#) (2017)
- [TouHou](#) ()
- [Mario Bros.](#) 1985
- [Devil May Cry](#) (2001)
- [Devil May Cry 3](#) (2005)
- [God of War](#) (2004)
- [Doki Doki Literature Club](#) (2017)
- [Beast Card Clash](#) (desde 2025, en desarrollo)
- [Club Penguin](#) ()
- [Animal Crossing](#) ()